# Phishing Twin: Architecture and Workflow

A read-only walkthrough of the phishing_twin repository: entry points, data and control flow, modules, state, configuration and dependencies.

**In one sentence:** a fictional hospital's 25 staff receive 40 simulated emails (20 phishing, 20 legitimate); a *digital twin* (`twin.py`) learns each person's click tendencies purely from observed Yes/No actions, either from a human tester in the Streamlit UI or from a scripted *simulated human* in the CLI test scripts.

Everything is plain Python and local JSON files. There is no database, server, API or queue.

## System diagram

```
OFFLINE DATA GENERATION
-----------------------
make_people.py  (random.seed(42))
      |  writes
      v
data/hospital_people.json            data/hospital_emails_mixed.json
25 staff: HR fields + "simulation"   40 emails: label, cue, target_role,
block with secret hidden_probs       sender, subject, body, red_flags
      |                                      |
      | persona.load_people()                | generator.EmailGenerator()
      v                                      v
 [Persona, Persona, ...]               emails  (list of dicts)
      |                                      |
      +-------------------+------------------+
                          |
      +-------------------+--------------------------+
      |                                              |
PATH A: STREAMLIT UI  (streamlit run app.py)    PATH B: CLI EXPERIMENTS  (python3 test_*.py)
----------------------------------------     ---------------------------------------------
Sidebar: pick staff member (person_id)          for each email:
Show next email from shuffled order               situation = make_random_situation()
      |                                           guess  = twin.predict(pid, email)
Human tester clicks  [Yes] / [No]                 action = SimulatedHuman.respond(email, sit)
      |  "click" / "ignore"                                  (reads hidden_probs + situation)
      v                                                      |
TwinPredictor.update(pid, email, action) <-- same twin.py --> TwinPredictor.update(...)
      |                                                      |
      +--> save_response() --> data/responses.json           v
      |                                           compare twin estimates vs hidden_probs
      v                                           print accuracy table to stdout
st.rerun(): next email, bar chart of
twin.get_all_estimates(pid)

NOT WIRED YET (stubs): main.py -> simulation.Simulation -> feedback.FeedbackModule
```

## 1. Entry points

| Entry point | How to run | What it does |
|---|---|---|
| `app.py` | `streamlit run app.py` | **Primary interactive entry.** A Streamlit web UI where a human labels emails Yes/No for a chosen staff member and watches the twin's estimates update live. |
| `make_people.py` | `python3 make_people.py` | Data generator. Builds the fictional hospital and staff and writes `data/hospital_people.json` (seeded, so the output is repeatable). |
| `test_twin.py` | `python3 test_twin.py` | One person, 30 random emails: predict, simulated human acts, twin updates. Prints guesses vs hidden_probs. |
| `test_twin_all.py` | `python3 test_twin_all.py` | Same loop for every person (100 emails each). Prints per-person average error. |

| Entry point | How to run | What it does |
|---|---|---|
| `test_situation.py` | `python3 test_situation.py` | Same person and email under calm vs chaotic situations, to show the situation nudges change click rates. |
| `test_persona.py`, `test_generator.py` | `python3 test_*.py` | Smoke checks: print loaded people, fetch one email for a role. |
| `main.py` | `python3 main.py` | **Stub.** `main()` is `pass`. Meant to wire up a full Simulation run. |

No `requirements.txt`, `pyproject.toml`, Dockerfile or other packaging exists. All scripts must be run from the repo root because data paths are relative.

# 2. Core data and control flow

## Primary action: a tester answers an email in the Streamlit UI

1. **App start / rerun.** Streamlit runs `app.py` top to bottom on every interaction. `get_people()` and `get_emails()` are wrapped in `@st.cache_data`, so the JSON files are read once per process.
2. **Load people.** `persona.load_people()` reads `data/hospital_people.json` and builds one `Persona` per entry in `data["people"]`. It maps `job_id` to `id` and unpacks the `simulation` block (role_group, awareness, workload, shift, hidden_probs).
3. **Load emails.** `EmailGenerator.__init__` reads `data/hospital_emails_mixed.json` and keeps `data["emails"]` (40 dicts).
4. **Session init.** If `st.session_state` has no `twin`, `reset_session()` creates a fresh `TwinPredictor`, shuffles the email indices into `order` and zeroes `pos` and `count`.
5. **Select person.** The sidebar `selectbox` returns `person_id` (for example `P05`). Switching person does *not* reset the email order. The twin keeps separate counters per person.
6. **Render email.** `email = emails[order[pos % len(order)]]`. The UI shows the sender name, subject and body. It does not show the label, cue or red_flags yet.
7. **User clicks Yes or No.** Yes maps to `"click"` and No maps to `"ignore"`.
8. **Twin update.** `twin.update(person_id, email, action)` picks a counter key: the email's `cue` for phishing, or `false_report` for legitimate mail. It increments `a` (clicked / reported) or `b` (did not).
9. **Persist.** `save_response(record)` reads `data/responses.json`, appends {time, person_id, person_name, email_id, cue, label, answer, action} and writes the whole list back.
10. **Advance and rerun.** `pos += 1`, `last` is stored for the feedback banner, then `st.rerun()`.
11. **UI update.** On rerun, the info box reveals whether the last email was phishing or legitimate and its cue. The right column draws `twin.get_all_estimates(person_id)` as a bar chart and table, where each value is `a / (a + b)`.

## Secondary flow: simulated experiment (CLI)

`test_twin.py`, `test_twin_all.py` and `test_situation.py` replace the human tester with `SimulatedHuman` (`human.py`):

- Phishing email: start from `persona.hidden_probs[cue]`, apply situation nudges, clamp to 0.02 to 0.95, then roll `random.random()`. The result is click or ignore.
- Legitimate email: roll against `hidden_probs["false_report"]`. The result is report or ignore.
- Situation nudges (`_adjust_for_situation`): night +0.10; email arrives during own shift +0.05; very busy +0.10; normal -0.05; billing cue to Finance & Billing +0.10.

- The script grades the twin by comparing `get_all_estimates()` with the true hidden_probs. Only the test harness reads hidden_probs, never the twin.

# 3. Component and module map

The repository is flat: Python modules at the root and a `data/` folder. There are no sub-packages.

| Module / folder | Single responsibility | Status |
|---|---|---|
| `persona.py` | `Persona` model plus `load_people()`: turns JSON people into objects. Holds visible HR fields and the secret `hidden_probs`. | Done |
| `generator.py` | `EmailGenerator`: loads the email corpus. `get_email(role)` returns a random email for a role or "all". The app uses the raw `.emails` list instead. | Done |
| `human.py` | `SimulatedHuman`: ground-truth behaviour. Decides click, report or ignore from hidden_probs and the situation. `learn()` is a stub. | Mostly done |
| `situation.py` | `make_random_situation()`: random time_of_day (day, evening, night) and busy_now (normal, busy, very busy) for each email delivery. | Done |
| `twin.py` | `TwinPredictor`: per-person Beta(1,1) counters for 5 cues plus false_report. `predict`, `update` and `get_all_estimates`. | Done |
| `app.py` | Streamlit UI and the human-in-the-loop labelling session. Persists answers to `responses.json`. | Done |
| `make_people.py` | Offline generator for the fictional hospital and its staff, including role baselines and awareness or workload adjustments. | Done |
| `simulation.py` | `Simulation`: will orchestrate the generate, predict, respond, feedback and record loop. | Stub |
| `feedback.py` | `FeedbackModule`: will produce feedback of different types (none, simple, detailed) after each email. | Stub |
| `main.py` | Will assemble all components and run a Simulation. | Stub |
| `test_*.py` | Runnable experiment and smoke scripts. These are not pytest tests and have no asserts. | Done |
| `data/` | All persistent state: people, emails and recorded responses. | - |
| `prompts.md`, `README.md`, *.pdf, *.pptx | Project notes and presentation material. | - |

## Data files

| File | Shape | Written by |
|---|---|---|
| `data/hospital_people.json` | {hospital, people[25]}. Each person has job_id, name, job_title, pillar, department, reporting_structure, ... and simulation{role_group, security_awareness, workload, shift, hidden_probs{urgency, authority, it_login, hr_payroll, billing, false_report}} | `make_people.py` |
| `data/hospital_emails_mixed.json` | {note, hospital, real_domain, counts{phishing: 20, legitimate: 20}, emails[40]}. Each email has id, label, cue, target_role, difficulty, sender_name, sender_email, subject, body, red_flags | Hand-authored (no generator in the repo) |

| File | Shape | Written by |
|---|---|---|
| `data/responses.json` | A JSON list of answer records, appended on every Yes/No. | `app.py` |

# 4. Key state and configuration

## State

- **Session state (Streamlit, in memory, per browser tab):** `twin` (a TwinPredictor), `order` (shuffled email indices), `pos`, `count` and `last`. This state is lost on page refresh or server restart. **Reset twin** rebuilds it.
- **Twin model state:** `TwinPredictor.counters` is a dict of person_id to cue to {a, b}. It is created lazily per person. It is never saved to disk, so the twin restarts from 0.5 every session.
- **Cached data:** `@st.cache_data` memoises the people and email lists for the lifetime of the Streamlit process.
- **Durable state:** only `data/responses.json`. It uses an unlocked read-modify-write, so this is fine for one user but two tabs answering at the same moment could lose a record. Reset twin does not clear the file, and the twin is not rebuilt from it on startup.
- **Ground truth:** `hidden_probs` is fixed in the people JSON. Situations are created fresh for each email in the CLI path. The Streamlit path does not use situations.

## Configuration

- No environment variables, `.env` file, config module or CLI flags. A grep for `os.environ` and `getenv` finds nothing.
- Settings are hard-coded constants: `RESPONSES_PATH` and `CUES` in `app.py`; default file paths in `load_people()` and `EmailGenerator()`; `NUM_EMAILS` and `random.seed(42)` in the test scripts; nudge sizes in `human.py`.
- Paths are relative, so the working directory must be the repo root.
- There is no `.streamlit/config.toml` in the repo. Only Streamlit's per-user `~/.streamlit/credentials.toml` exists, which belongs to Streamlit, not this project.

# 5. External dependencies and services

| Dependency | Used in | Purpose |
|---|---|---|
| `streamlit` | `app.py` | Web UI, session state, caching, charts. Optional `watchdog` only speeds up file-change reloads. |
| `pandas` | `app.py` | DataFrame for the estimates bar chart and table. |
| Python stdlib (`json`, `random`, `os`, `datetime`) | all modules | File I/O, randomness, timestamps. |

- **Databases:** none. Persistence is local JSON files in `data/`.
- **External APIs:** none. No HTTP clients and no LLM SDKs. Email content is static JSON.
- **Queues or background jobs:** none. Everything is synchronous and single-process.
- **Network:** only the local Streamlit server (default `localhost:8501`). The URLs in email bodies use the `.example` TLD and are never fetched.

# Observations and gaps

- **false_report cannot rise in the UI.** The app only sends `"click"` or `"ignore"`. For legitimate emails the twin only counts `"report"` as a, so every answer pushes false_report towards 0. A Yes on a legitimate email is

recorded as a non-report.

- **Role targeting is unused in the UI.** Each email has `target_role` and `get_email(role)` can filter on it, but `app.py` shows every person all 40 emails.
- **Situation layer is CLI-only.** `situation.py` affects SimulatedHuman but is not shown or recorded in the Streamlit flow.
- **Twin is not persisted.** Learned counters disappear on refresh. `responses.json` has enough data to rebuild them, but nothing replays it.
- **Docs drift.** `README.md` says there are 30 staff, but the data has 25 after commit `8fef1ed`. The README also does not mention `app.py` or `situation.py`.
- **Pipeline not assembled.** `main.py`, `simulation.py`, `feedback.py` and `SimulatedHuman.learn()` are placeholders.